

Tackling Domain Shift using Fast R-CNN and ResNet18

Alessandro Nicosia

ALESSANDRO.NICOSIA@STUDIUM.UNICT.IT

1. Introduction

My project aims to tackle the challenge of domain shift in an object classification problem. The training dataset consists of images of 9 different objects, each clearly visible in the foreground against various backgrounds. However, class 0 is an exception: its images consistently feature the same background—a backyard with grass, shrubs, and a tree. The testing set includes images of the same 9 objects, but all of these images share the background seen in class 0. This creates a significant domain shift challenge, as the neural network might incorrectly classify all testing images as class 0 due to the shared background.

To address this challenge, I propose a two-step approach utilizing Fast R-CNN and ResNet18. First, I use pre-trained Fast R-CNN to generate accurate bounding boxes that isolate the objects from their backgrounds. I crop the objects and place them onto a uniform background at random positions. This preprocessing step is applied to both the training and testing datasets to minimize the influence of the original backgrounds. With these modified datasets, I employ non-pretrained ResNet18 for the classification task. ResNet18, with its architecture based on residual blocks, is well-suited for handling complex patterns and demonstrates robustness against overfitting, making it an ideal choice for this problem.

2. Model Description

In my approach, I used ResNet-18 ([He et al., 2015](#)) and Fast R-CNN ([Kingma and Ba, 2015](#)) to tackle the domain shift problem. ResNet-18 is a deep convolutional neural network designed to address the vanishing gradient problem through the use of residual blocks. It consists of 18 layers, including 8 residual blocks, where each block has two convolutional layers followed by batch normalization and ReLU activation. The key innovation in ResNet-18 is the identity shortcut connection, which allows the input of a layer to bypass one or more layers and be added directly to the output. This skip connection enables the training of much deeper networks by preserving gradient flow, making the optimization process more efficient. The architecture includes an initial convolutional layer with a large kernel size, followed by a max pooling layer and a series of residual blocks that progressively decrease the spatial dimensions while increasing the depth of the feature maps. Finally, it ends with a global average pooling layer and a fully connected layer that outputs class probabilities.

I utilized Faster R-CNN for the object detection task, combining a ResNet-50 backbone with a Feature Pyramid Network (FPN) to enhance detection accuracy across various object scales. The ResNet-50 backbone extracts rich feature representations through its 50-layer deep residual network, while the FPN builds a pyramid of these features at different resolutions. In the Faster R-CNN framework, these multi-scale feature maps are utilized

by the Region Proposal Network (RPN) to generate region proposals—potential bounding boxes that might contain objects. The RPN proposes regions of interest (ROIs) that are likely to contain objects that are then warped into a fixed size and fed into a regressor. The regressor refines the bounding box coordinates to more accurately fit the detected objects.

In the training process I splitted the training dataset into training and validation sets to monitor model generalization. I utilized the Adagrad optimization algorithm, initializing it with a learning rate of $\eta = 0.001$, chosen empirically through experimentation. Adagrad changes the learning rate for each parameter based on past gradients, helping to efficiently navigate the loss during training. To evaluate model performance, I employed the Cross Entropy Loss function, suitable for multi-class classification tasks, defined as:

$$\text{CrossEntropyLoss}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \log(\hat{y}_i)$$

where $\mathbf{y}$ represents the true labels and $\hat{\mathbf{y}}$ denotes the predicted probabilities from the model. I implemented early stopping to monitor the validation loss; training halts if the validation loss does not improve after a specified number of epochs, preventing overfitting and ensuring that the model generalizes well to unseen data.

3. Dataset

The dataset is divided into a training set and a testing set. The training set consists of 1,800 images, each containing a single object from one of 9 possible classes, with annotated bounding box coordinates. The testing set includes 900 images, each also containing one object from these classes, but without any provided bounding boxes. In the training set, each object class is shown against various backgrounds, except for classes 0 and 9, which always have the same background. The provided bounding boxes are imprecise, as they encompass not only the object but also the hand and forearm of the person holding it, along with a significant portion of the background. In the testing set, all images feature the same background as class 0 in the training set. One of the classes in the dataset was entirely created by me. Specifically, I collected all 300 images for this class and manually annotated the bounding boxes. The testing images for this class feature a background similar to that of class 0.

To address the domain shift problem, I preprocessed the 2,700 images in two steps. First, I improved the accuracy of the bounding boxes using Fast R-CNN, ensuring that the boxes captured only the object, with minimal background and a few fingers. Approximately 3% of the testing images and 3.3% of the training images still had imprecise bounding boxes, similar to those in the original dataset.

After refining the bounding boxes, I cropped the images to these coordinates and pasted the cropped images onto a uniform green background. Then, I applied data augmentation techniques: for both testing and training, I standardized the images using mean and standard deviation values (mean: [0.485, 0.456, 0.406] and standard deviation: [0.229, 0.224, 0.225]), and for training only, I randomly rotated the images by up to 30 degrees.

4. Training details and procedure

1. Evaluation Metrics

The project involves a multi-class classification task. For this reason, the chosen metric for evaluating the model’s performance is accuracy, which represents the ratio of correctly predicted instances to the total instances in the dataset. This makes it an intuitive and effective way to assess how well the model is performing across multiple classes.

2. Training procedure

The model was set to be trained for 100 epochs, however early stopping was often triggered between epochs 79 and 85. Early stopping was implemented to monitor the validation loss at each epoch. If the validation loss decreased, training continued and the model’s weights and optimizer state were saved in a file. If the validation loss did not decrease for seven consecutive epochs, the training process was stopped to prevent overfitting. The number of consecutive epochs without improvement in the validation loss was chosen based on many previous experiments with also different approaches.

The original training dataset was divided into training and validation sets using an 80/20 split, resulting in 1,440 training images and 360 validation images. The data was split using a stratified method to ensure that each class was proportionally represented in both sets. The batch size was set to 128 for both validation and training sets.

3. Training details

In this training setup, I use the Adagrad (Adaptive Gradient Algorithm) optimizer with an initial learning rate of 0.001, determined through experimentation. Additionally, I integrate the Reduce-On-Plateau scheduler, which dynamically adjusts the learning rate during training based on validation loss improvements. For weight initialization of the ResNet-18 model, I employ PyTorch’s default initialization. Although I have also tested He initialization, which offers a better starting point on average, I observed that both methods result in similar performance (see Fig. 1).

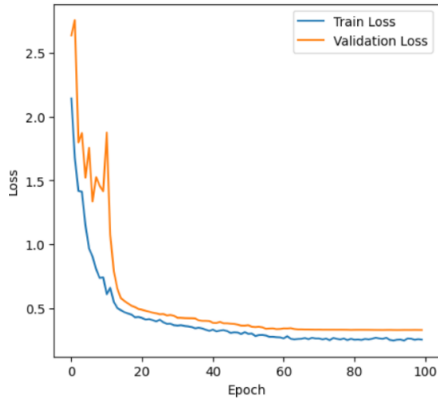
5. Experimental Results

The model achieved an accuracy of 92.50%, the highest value among various training attempts. In each training session, the final loss function value ranged between 0.28 and 0.25. In the best training process (see Fig. ??), early stopping was triggered at epoch 79. By epoch 30, the validation loss had decreased to 0.2874, with an accuracy of 92.19% on the validation set. Between epochs 30 and 79, the improvements were minimal, with the validation loss decreasing by just 0.0192 and accuracy increasing by only 0.31 percentage points. On the test set, I achieved 89.3% accuracy. Through experimentation, I determined

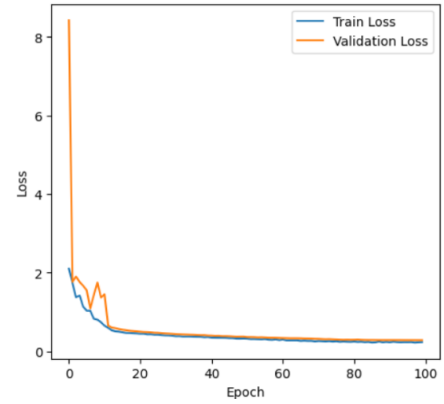
that the optimal starting learning rate for my model is 0.001. Initially, I tried a learning rate of 0.0001 (see Fig. 2(b)), but it was very slow. It required over 100 epochs to achieve the same loss function value that a starting learning rate of 0.001 reached in just 30 epochs. Additionally, I experimented with a learning rate of 0.01 (see Fig. 2(a)), which proved to be as slow as the 0.0001 learning rate, reaching a similar validation loss value after 100 epochs. In this experiment, the validation loss value in the first epoch consistently exceeded 2000. Therefore, the graph does not show the loss value for the first epoch. Furthermore,

I replaced Adagrad with Stochastic Gradient Descent (SGD) to evaluate their performance differences. For SGD, I set the learning rate to 0.1 because the default value of 0.001, which worked well with Adagrad, did not facilitate convergence in the loss function when using SGD. After 100 epochs of training with SGD, the minimum value achieved for both the training and validation loss was approximately 0.58 (see Fig. 3(a)). This value is significantly higher than the loss observed when using Adagrad, indicating poorer performance. The accuracy on the testing set was approximately 79%. These results highlight that Adagrad is more effective for my specific task compared to standard SGD.

For comparison, I experimented with a pretrained ResNet18 model. I conducted two experiments: fine-tuning only the classifier and Layer4 (see Fig. 5), which comprises the last two residual blocks, and fine-tuning only the classifier (see Fig. 6). Both approaches yielded suboptimal results compared to training the entire network from scratch, achieving a testing accuracy of around 73% in both cases.

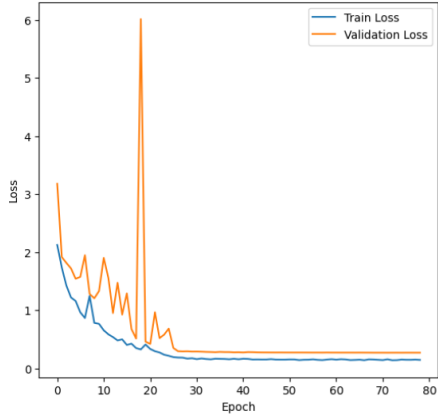


(a) He distribution

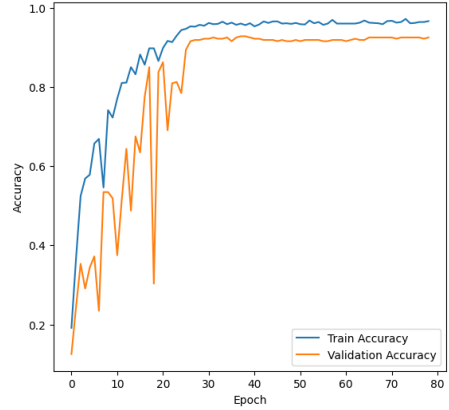


(b) Default initialization

Figure 1: Comparison between He distribution and Default initialization.

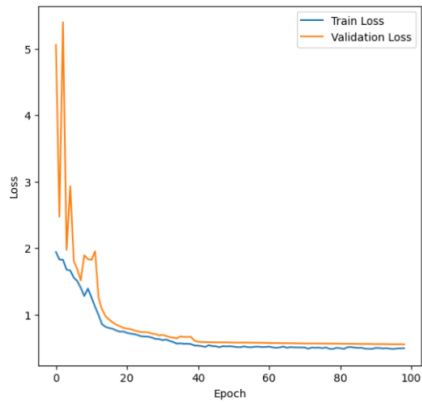


(a) Loss Curves

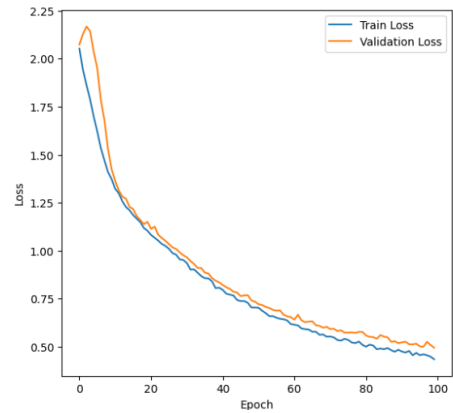


(b) Accuracy Curves

Figure 2: Loss and Accuracy curves for the optimal training process.

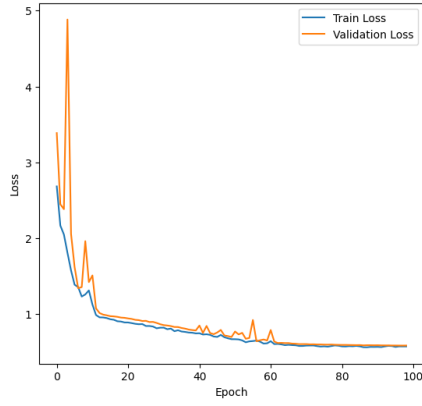


(a) Starting Learning Rate: 0.01

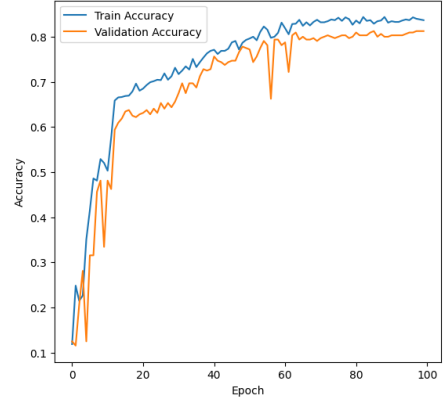


(b) Starting Learning Rate: 0.0001

Figure 3: Learning Rate experiments with two different initial rates: 0.01 and 0.0001.

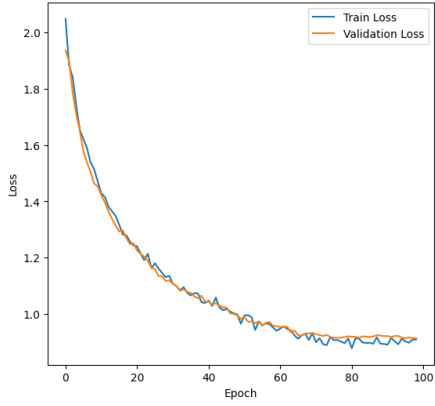


(a) Loss Curves

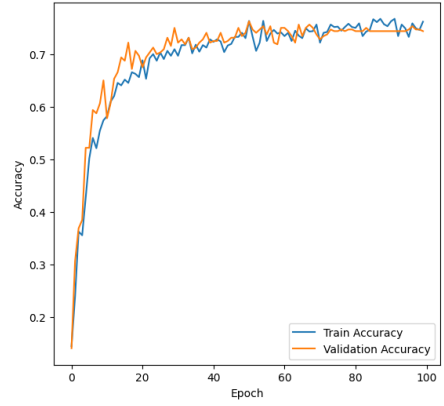


(b) Accuracy Curves

Figure 4: Performance of Stochastic Gradient Descent: Loss and Accuracy Curves.

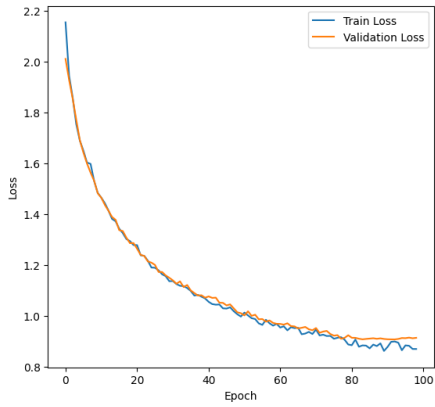


(a) Loss Curves

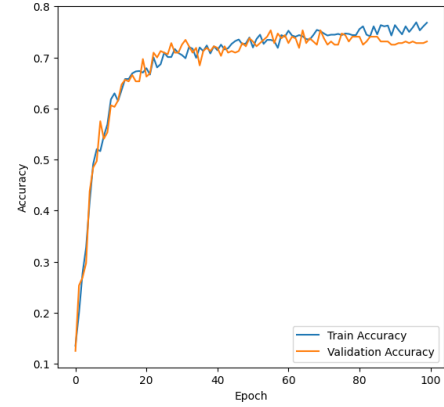


(b) Accuracy Curves

Figure 5: Performance of Pre-Trained ResNet-18 fine-tuning Classifier and Layer4.



(a) Loss Curves



(b) Accuracy Curves

Figure 6: Performance of Pre-Trained ResNet-18 fine-tuning only the Classifier.

6. Code and Demo

You can find the repository for my project at the following link: <https://github.com/alenicasia31/Machine-Learning-Project>. The repository contains three main folders:

- **Image Processing Script:** This folder contains the script used to apply Fast R-CNN to the images.
- **Project Notebook:** In this folder, you will find the notebook with the code used to train and test the ResNet18 model on the processed dataset.
- **Demo:** This folder includes the demo script, a video demonstrating how the demo works, and two empty folders. These folders need to be filled with two datasets (the preprocessed and the original one) and the model weights, which can be downloaded from the Google Drive links provided in the ReadMe file.

After filling the folders, you need to update the directory paths in the `demo.py` file as shown in the video. Once this is done, you can run the demo, which will guide you through a simple, user-friendly interface.

7. Conclusions

In this project, I addressed domain shift in object classification by using a combination of Fast R-CNN and ResNet18. My approach involved refining bounding boxes using a pre-trained network to better focus on the objects and standardizing the backgrounds across all images, effectively minimizing the impact of background variations that could cause misclassification. Through experimentation, I demonstrated that robust preprocessing, appropriate hyperparameter tuning and the choice of optimization strategies, such as Adagrad, are crucial for improving model performance and convergence.

To improve the proposed method, one could start by applying a more effective normalization technique to standardize the image backgrounds, thereby reducing the impact of background variations and potentially eliminating the need for bounding boxes. This would enable the model to focus more on the objects of interest, enhancing its robustness to domain shifts. Additionally, incorporating Vision Transformers (ViTs) could further improve performance. ViTs use a self-attention mechanism that allows them to identify and focus on the most relevant parts of an image, regardless of their spatial location. This ability could reduce the model's reliance on data augmentation and make the classification process more efficient and accurate.

References

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. URL <http://arxiv.org/abs/1512.03385>.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2015. URL <http://arxiv.org/abs/1412.6980>.